

C# Self-Study Report

C# Fundamentals - Self Study

1. Compiled vs Interpreted Languages

- **Compiled:** Source code is translated by a compiler before execution.
- **Interpreted:** Source code is executed through an interpreter at runtime.
- **C#:** C# is compiled into **Intermediate Language (IL)**, then the **CLR/JIT** converts the IL into native machine code at runtime.

C# → Compiler → IL → CLR/JIT → Machine Code

2. C# Managed Code

Managed Code is code that runs under the **CLR (Common Language Runtime)**.

The CLR provides:

- Automatic memory management.
- Garbage Collection.
- Exception handling.
- Type safety.

Example: C# applications running on .NET are managed by the CLR.

3. Struct vs Class

Both struct and class can contain fields, properties, methods, and constructors.

Struct	Class
Value Type	Reference Type
Copies the value when assigned	Copies the reference
Cannot inherit from another class/struct	Supports class inheritance
Usually suitable for small data values	Suitable for objects and complex data

Example:

```
struct Point { public int X; }
```

```
class Person { public string Name; }
```

Key idea: Struct behaves like a class in some programming features, but fundamentally it is a **Value Type**, while a class is a **Reference Type**.

4. Type Conversion in C#

Type	Example	Meaning
Implicit	<code>double x = 10;</code>	Automatic conversion
Explicit	<code>int x = (int)10.5;</code>	Manual conversion, possible data loss
Parse	<code>int.Parse("10")</code>	Converts a string
Convert	<code>Convert.ToInt32("10")</code>	General-purpose conversion

Example:

```
double x = 10.5;
```

```
int y = (int)x; // 10
```

Explicit casting is required because converting double to int can lose the decimal part.

Conclusion

This self-study helped me understand the basics of **C# compilation, managed code, structs vs classes, and type conversion**, which are important concepts for working with C# and .NET.